

Real-Time Sentiment Streaming Engine (Go + Rust)

This document talks about a prototype system made to show how Golang can be used in real situations. And Rust working together in a backend processing pipeline. The project simulates real-time sentiment analysis engine that looks at streaming text messages and sorts them as Positive, Negative, or Neutral?

Project Objective

The aim of this project was to create a small, realistic system that shows:

- Backend processing running at the same time using Golang
- Rust offers fast and efficient text processing that handles tasks smoothly and quickly.
- Basic Natural Language Processing sentiment scoring involves analyzing text to figure out if the overall feeling is positive, negative, or neutral. It's a simple way to get a sense of how people feel based on their words.
- Cross-language system integration means making different systems that speak different programming languages work together smoothly.
Streaming style processing with worker pools

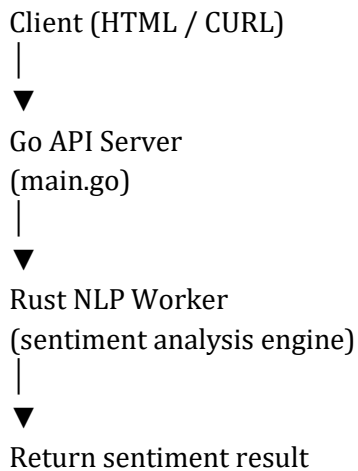
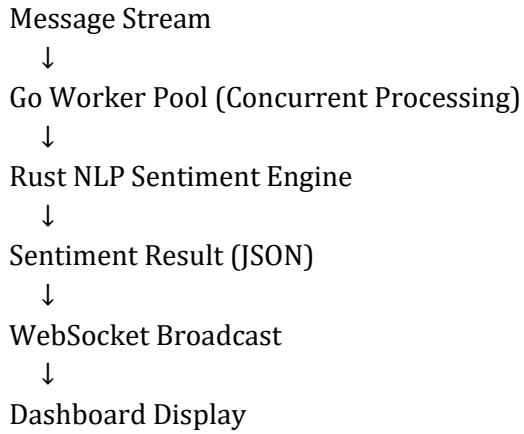
Architecture Overview

The system has three main parts:

1. Set up a streaming service with an API and a worker pool.
2. Rust NLP Sentiment Engine
3. Web Dashboard for live visualization

The Go service either creates or gets messages and then spreads them out among worker threads. Each worker sends the text over to the Rust engine. Rust takes the text and gives back the sentiment score is sent straight to the dashboard in real time.

System Flow



Golang Responsibilities

Golang is used as the orchestration and streaming layer. Its responsibilities include:

- Running the HTTP API server
- Managing worker pools using goroutines
- Distributing incoming messages to workers
- Calling the Rust sentiment engine
- Broadcasting results to the dashboard using WebSockets
- Maintaining live sentiment statistics

Rust Responsibilities

The Rust module acts as the sentiment analysis engine. Rust was chosen for its performance and memory safety. The module:

- Accepts JSON input containing text
- Normalizes the text
- Compares words against positive and negative dictionaries
- Calculates a sentiment score
- Returns structured JSON containing sentiment classification

Sentiment Scoring Method

The system works by following a set of rules to process language. Here is a list of positive and negative words. Maintained each time a positive word shows up, the score goes up, and each negative word brings it down, word decreases it.

Example:

Text: 'This phone is amazing but battery is bad'

Positive words: amazing

Negative words: bad

Score = +1 - 1 = 0

Sentiment = Neutral

Streaming Processing

Streaming Processing is a way to handle data as it comes in, rather than waiting for it all to be collected first. It lets you work with information in real time, which is handy when you need quick results or want to react to events immediately. This approach is common in things like monitoring sensors, handling live video feeds, or tracking online activity as it happens. It's all about dealing with data right away, instead of later.

The Go service keeps a streaming pipeline going by constantly creating messages and putting them in a job queue. Several workers handle messages at the same time. This architecture is similar to real-world event processing systems that manage large amounts of data. lots of social media posts or customer reviews

Dashboard Visualization

A simple HTML dashboard links up with the Go service using a WebSocket connection. As messages come in, the server sends sentiment results out to all connected clients.

The dashboard shows:

- Live message feed
- Sentiment classification
- Positive / Negative / Neutral statistics

Skills Demonstrated

This project demonstrates several practical software engineering skills:

- Backend development with Golang
- Systems programming with Rust
- Natural Language Processing fundamentals
- Concurrent programming using worker pools
- Cross-language service integration
- Real-time data streaming architecture

Setup Instruction

- 1) Build the Rust NLP Worker (first time only)

```
cd rust-nlp-worker  
  
cargo build
```

Note: This will generate the executable used by the Go service. If the binary is already included in the project, this step can be skipped.

- 2) Start the Go API Server

```
cd go-api  
go run main.go
```

- 3) Open the Dashboard

Open index.html file which is dashboard/index.html

Conclusion

By combining these languages, you get a system that's easier to manage and can handle large amounts of data without slowing down. The key is to let each language do what it does best and connect them smoothly, often with APIs or data formats everyone understands. This way, the whole system stays scalable and works well as the data grows systems. Golang offers concurrency and orchestration, while Rust brings efficiency and safe text processing. Together, they create a system that can manage streaming sentiment, analysis workloads